

PROJECT REPORT

LIBRARY MANAGEMENT SYSTEM

Submitted in partial fulfillment of Python Essentials

Submitted by:	Jiya Jasuja [26MIM10238]
Project Title:	Library Management System
Language Used:	Python 3.x
Submitted to:	Dr. Gunjan Ansari
Subject:	Python Essentials

TABLE OF CONTENTS

1. Problem Statement	Page 2
2. Introduction	Page 2
3. Objectives & Target Audience	Page 3
4. Functional & Non-Functional Requirements	Page 3
5. System Architecture & Data Structure	Page 4
6. Tools and Technologies Used	Page 4
7. Module Description	Page 5
8. Code Walkthrough & Sample Output	Page 5
9. Testing, Limitations & Future Enhancements	Page 6
10. Conclusion	Page 6

1. PROBLEM STATEMENT

Small libraries, community groups, and personal collectors often face significant friction in cataloging books, maintaining accurate inventory counts, and managing book issues and returns. Manual record-keeping on paper is error-prone, labor-intensive, and lacks quick search capabilities. While enterprise database systems exist, they are excessively complex, require third-party packages, and demand heavy installation overhead unsuitable for lightweight standalone personal use or introductory programming education. Thus, there is a clear requirement for a self-contained, lightweight, console-based library utility written in pure Python that handles inventory tracking, search indexing, and borrowing transactions seamlessly without external dependencies.

2. INTRODUCTION

The Library Management System is a beginner-friendly console application built entirely in pure Python without external dependencies or databases. It simulates core daily operations of a small library via a text menu, tracking titles, authors, copy counts, and borrower assignments during runtime.

3. OBJECTIVES & TARGET AUDIENCE

Objectives: To design a clean procedural system for managing books, supporting record additions, keyword searching, book issuing/returning, and deletion using fundamental Python lists and dictionaries.

Target Audience: Beginner Python students learning procedural programming and data structures, and small community organizers needing offline command-line tracking.

4. FUNCTIONAL REQUIREMENTS

- **Book Registration (add_book):** Validates and registers new books with a title, author, and positive copy count, assigning an incremental unique ID.
- **Catalog Viewing (view_books):** Displays a formatted table listing book IDs, titles, authors, and real-time available copy counts.
- **Catalog Search (search_book):** Case-insensitive keyword search matching against book titles and author names.
- **Book Issuing (issue_book):** Issues a book copy to a borrower after verifying availability and preventing duplicate checkouts.
- **Book Return (return_book):** Processes book returns by removing borrowers from active issue records.
- **Book Deletion (delete_book):** Deletes books via ID, blocking deletion if active copies remain checked out.

NON-FUNCTIONAL REQUIREMENTS

- **Portability:** Operates on standard Python 3 runtime environments with zero third-party package installation overhead.
- **Performance:** Instantaneous command execution for standard in-memory library catalogs.
- **Robustness & Validation:** Input error handling prevents runtime application crashes from invalid numeric or empty submissions.

5. SYSTEM ARCHITECTURE & DATA STRUCTURE

The application follows a modular procedural software architecture. All book data is stored in runtime memory inside a primary Python list named `books`, where each element is a structured dictionary managed by an auto-incrementing global counter `next_id`.

Field Name	Data Type	Description
<code>id</code>	Integer	Unique auto-assigned sequential identifier for each book record
<code>title</code>	String	Title of the book
<code>author</code>	String	Name of the book's author
<code>copies</code>	Integer	Total physical copies owned by the library
<code>issued_to</code>	List of Strings	Active borrower names currently holding a book copy

Dynamic Availability Formula: Available copies are computed dynamically on demand as `copies - len(issued_to)` to maintain absolute consistency.

6. TOOLS AND TECHNOLOGIES USED

Component	Details
Programming Language	Python 3.x
Interface Type	Command Line Interface (CLI)
External Libraries	None (pure built-in Python features)
Data Storage	In-memory (Python list of dictionaries)
Development Environment	Any Python-compatible IDE or Terminal

7. MODULE DESCRIPTION

- `add_book()`: Prompts for book details, validates inputs, and appends the record.
- `view_books()`: Formats and displays the complete inventory table.
- `find_book_by_id()`: Helper function retrieving book dictionaries by ID.
- `search_book()`: Performs case-insensitive substring searching across title and author.
- `issue_book()`: Handles checkout verification and borrower registration.
- `return_book()`: Processes returns and updates active issuance lists.
- `delete_book()`: Removes books safely if no active issues exist.

8. CODE WALKTHROUGH & MENU DISPATCH

Dictionary-based function mapping avoids long conditional chains:

```
menu_actions = { "1": add_book, "2": view_books, "3": search_book, "4": issue_book, "5":  
return_book, "6": delete_book } while True: print_menu() choice = input("Enter your choice:  
").strip() if choice == "0": break action = menu_actions.get(choice) if action: action() else:  
print("Invalid choice.")
```

9. TESTING & LIMITATIONS

Testing: Manually verified with valid and invalid inputs across all modules. Empty fields, negative copy counts, duplicate borrowers, and unauthorized deletions are correctly intercepted and handled.

Limitations: Data is volatile (in-memory only and lost on exit); no persistent database; lacks overdue fines and user authentication roles.

10. FUTURE ENHANCEMENTS

- Integrate JSON or SQLite file-based persistence for data retention.
- Add due dates and automatic overdue fine calculations.
- Implement user role authentication (Librarian vs. Member).
- Develop a graphical user interface (GUI) or web frontend.

11. CONCLUSION

The Library Management System successfully demonstrates the application of core Python concepts—functions, dictionaries, lists, and procedural control flow—into a practical, menu-driven inventory solution. Its robust modular architecture establishes an ideal foundation for future enhancements and database integrations.